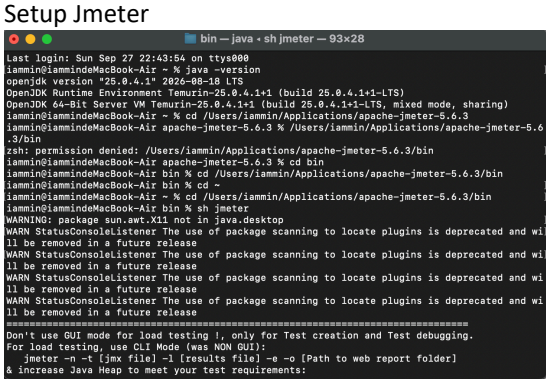
Tester (QA) Week 4

Monday, September 21, 2026 19:19

Week 4 — End-to-End Frameworks & Non-Functional Testing

- **Courses:**
  - Performance and Load Testing with Apache JMeter
  - BDD Testing with Cucumber and Gherkin (Java or Python)
  - Security & Vulnerability Testing Fundamentals (OWASP Top 10)
    - Web security testing basics (OWASP Top 10 / OWASP ZAP)
- **Key Concepts:** Load profiling, ramp-up time vs. throughput, virtual user concurrency, headless browser automation, parameterized stress testing.
- **Practice:**
  - Create a JMeter test plan executing a 50-user load test against public REST endpoints; analyze throughput, error rates, and response latencies.
  - Convert your Selenium automation suite into a BDD test harness using Cucumber.
  - Selenium tests into a Page Object Model framework, and write 3 BDD scenarios in Gherkin for the same feature. Capstone: present your full 5-week test suite, CI pipeline, and bug log to the team

Setup Jmeter

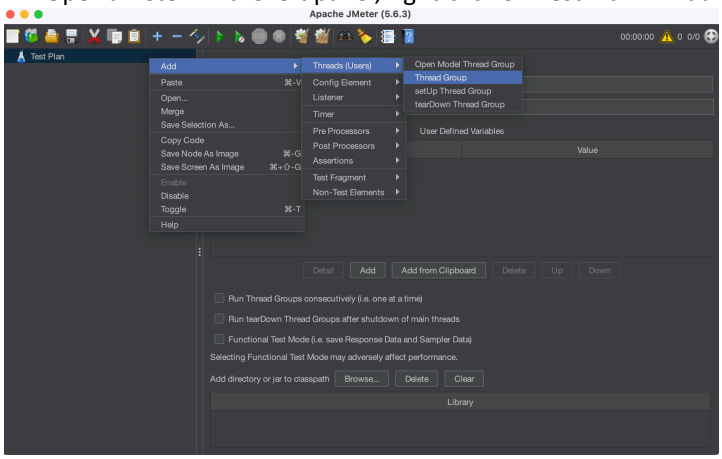


If already installed, just input command "cd(space, and drag the bin folder within the jmeter folder into the teminal)", then click Enter, Jmeter GUI opened

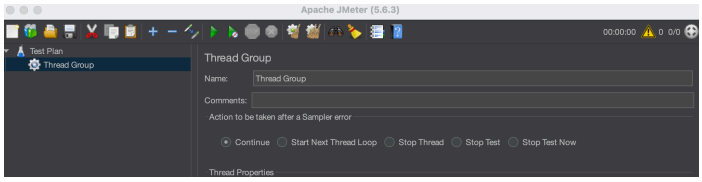
Task1: Execute a 50-User Load Test using Apache JMeter

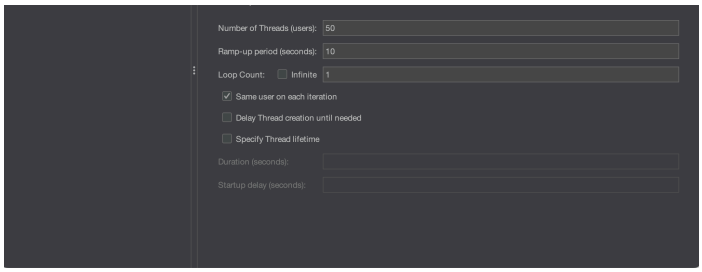
Step-by-Step Setup:

1. Open JMeter. In the left panel, right-click on Test Plan -> Add -> Threads (Users) -> Thread Group.

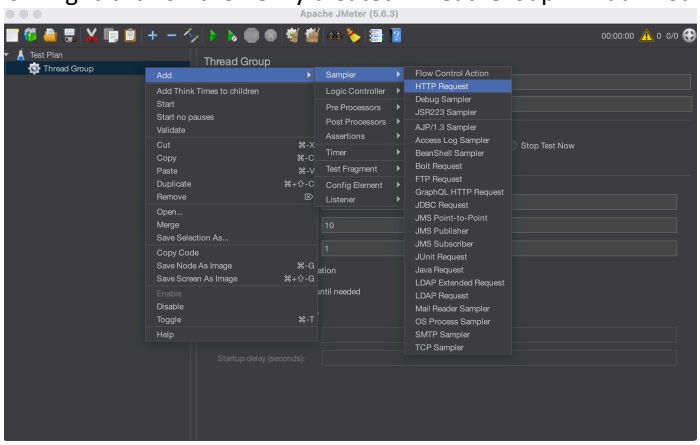


2. In the **Thread Group** configuration panel, set **Number of Threads (users)** to 50 and the **Ramp-up period (seconds)** to 10 (This means all 50 virtual users will be started up over a 10-second window).

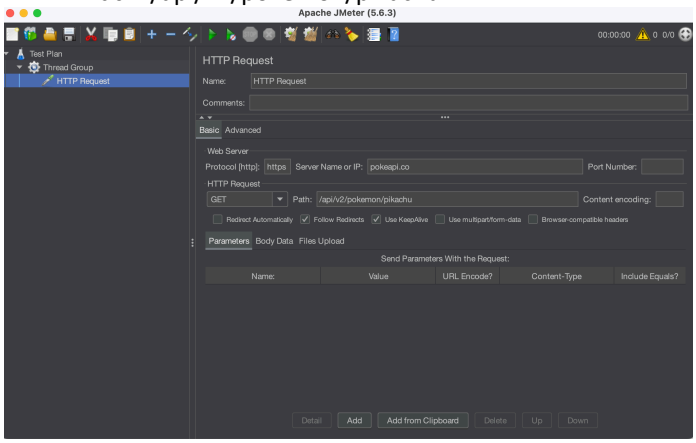




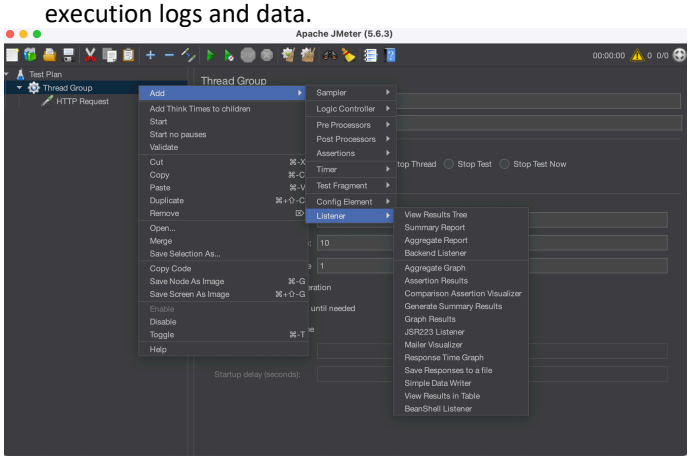
3. Right-click on the newly created Thread Group -> Add -> Sampler -> HTTP Request.



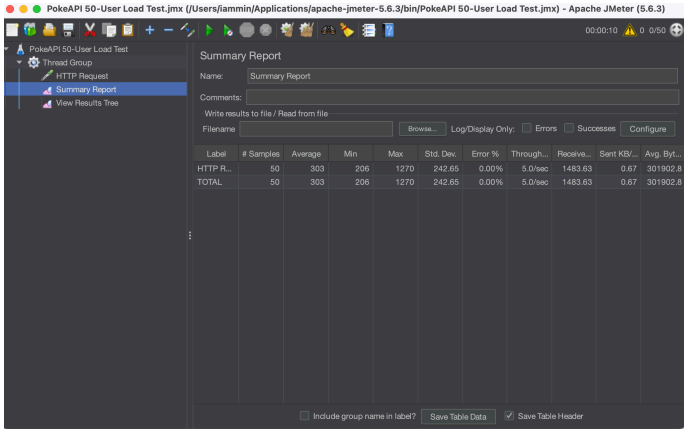
4. In the **HTTP Request** configuration panel, set up the endpoint details:
   - **Protocol:** https
   - **Server Name or IP:** pokeapi.co
   - **HTTP Request Method:** GET
   - **Path:** /api/v2/pokemon/pikachu



5. Right-click on the Thread Group -> Add -> Listener. Add both **Summary Report** and **View Results Tree** so you can view the execution logs and data.

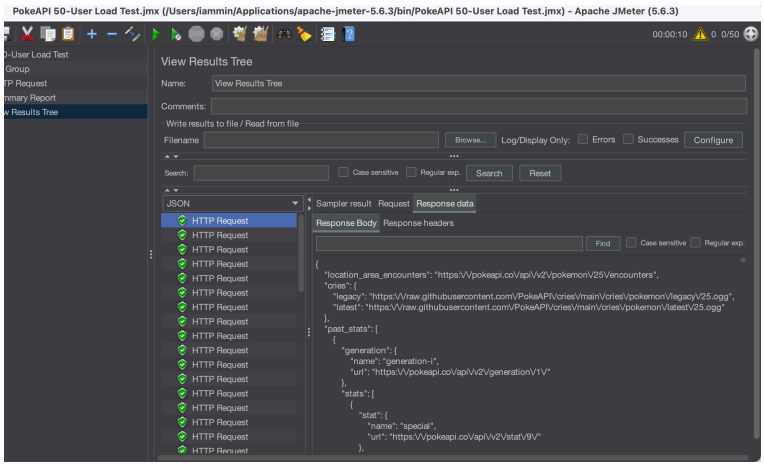


6. Click the green **Start** button on the top toolbar to execute the test.



Performance Test Results Summary:

- **Throughput:** The system successfully handled an average throughput of 5.0 requests per second against the PokéAPI endpoint. This indicates that the server effectively processed an average of 5 requests every second.
- **Error Rate:** The test completed with an error rate of 0.00%, confirming stable API endpoint availability under a 50-user concurrent load. This demonstrates that all 50 requests successfully retrieved data without encountering any 404 (Not Found) or 500 (Internal Server Error) issues.
- **Response Latency:** The average response time was 303 ms, with a maximum response latency peaking at 1270 ms and a minimum response time of 206 ms. Overall, an average response time of 303 ms represents solid performance for a public API. The slowest individual request took 1.27 seconds (1270 ms), which remains well within acceptable operational thresholds.



View Results Tree Analysis:

- **Execution Validation:** The View Results Tree listener confirms a 100% success rate, displaying 50 individual HTTP Request samples all marked with green indicators. This verifies that every concurrent thread successfully reached the server without network timeouts or connection rejections.
- **Data Integrity Check:** Inspection of the "Response data" tab confirms the endpoint returned the correct JSON payload. The response body accurately displays the requested Pokémon's attributes, including specific nested objects like location_area_encounters, cries, and past_stats, proving the API backend processed the GET requests correctly and returned valid, well-formatted data structures.

Performance Testing is a broad category of non-functional testing that ensures software applications perform well under their expected workload. It encompasses several specific types of testing:

1. **Load Testing:**
   - **Objective:** To verify that the system behaves normally and maintains acceptable performance metrics (such as response time, throughput, and error rates) under the expected normal concurrent user load.
   - **Application:** This involves simulating an expected normal workload, such as configuring 50 concurrent virtual users to hit an endpoint over a set timeframe. The goal is to validate that under this standard pressure, the system maintains stable performance, such as a 0% error rate and healthy response times, before attempting to find its breaking point.

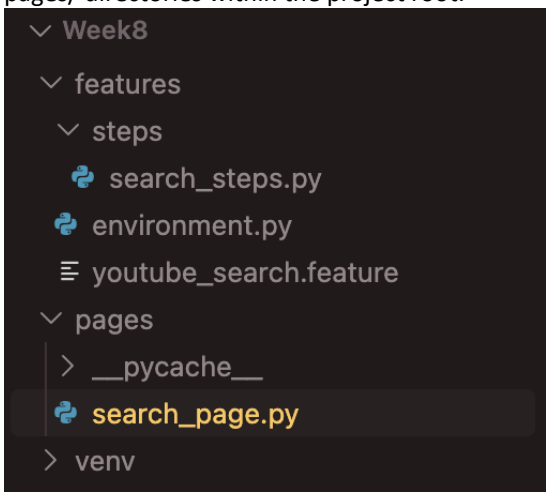
2. **Stress Testing:**
   - **Objective:** To find the system's **breaking point**.
   - **Distinction:** If you were to increase the Threads to 5,000 or 10,000 and continually ramp up the pressure until the server starts throwing 500 Internal Server Errors (crashing) or response times degrade into hundreds of seconds, you would be conducting a Stress Test. The goal is to see the absolute maximum capacity the system can handle before it fails.

3. **Spike Testing:**
   - **Objective:** To evaluate how the system handles **sudden, extreme, and unexpected** spikes in traffic (e.g., a Black Friday flash sale).
   - **Distinction:** If you changed the Ramp-up period to 0 seconds—meaning all 500 users hit the endpoint simultaneously in a single massive wave—rather than a gradual 10-second slope, you would be executing a Spike Test.

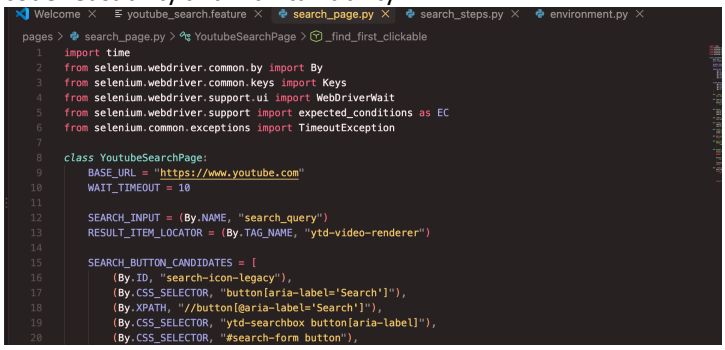
4. **Endurance Testing (or Soak Testing):**
   - **Objective:** To determine if the system can sustain the expected normal load continuously over an **extended period of time** to identify issues like memory leaks or database connection exhaustion.
   - **Distinction:** If you set the test to 50 users but configured JMeter to run continuously for 24 or 48 hours instead of just a few seconds, that would be an Endurance Test.

Task2:
Convert your Selenium automation suite into a BDD test harness using Cucumber.
Selenium tests into a Page Object Model framework, and write 3 BDD scenarios in Gherkin for the same feature.

Step 1: Setting up the BDD Project Directory Structure
To implement the Cucumber/Behave framework, a strict, standard BDD directory structure is required. This separates the test logic into test scenarios (Feature files), step definitions, and the Page Object Model (POM). I established features/ and pages/ directories within the project root.



Step 2: Refactoring with the Page Object Model (POM)
I refactored the procedural Selenium test code into a Page Object Model (POM) design pattern. Within the pages/search_page.py file, I centralized all web element locators and underlying interaction methods, significantly improving code reusability and maintainability.

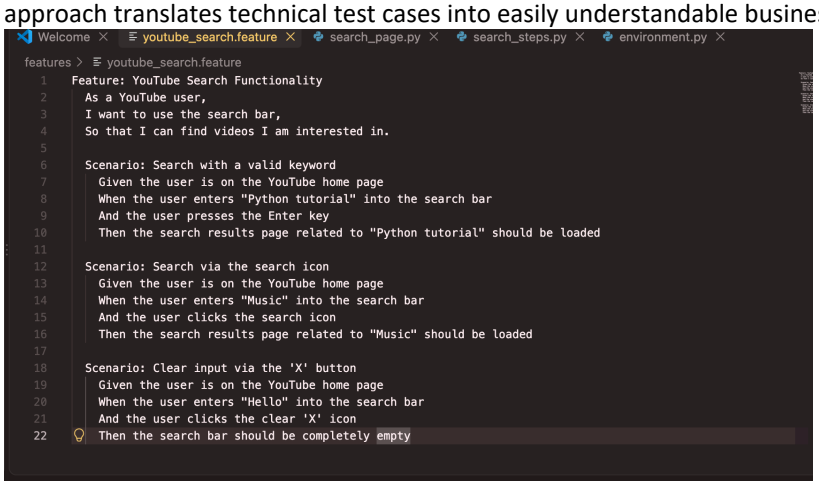


```
    ]

    CLEAR_BUTTON_CANDIDATES = [
        (By.XPATH, "/html/body/ytd-app/div[1]/div[2]/ytd-masthead/div[4]/div[2]/yt-searchbox/div[1]/div/div
```

Step 3: Writing Test Scenarios using Gherkin Syntax

I wrote plain-text test scenarios using Gherkin syntax (Given/When/Then) in the features/youtube_search.feature file. This approach translates technical test cases into easily understandable business requirements for non-technical stakeholders.



Step 4: Implementing Step Definitions

In the features/steps/search_steps.py file, I implemented the step definitions (often referred to as "glue code"). This crucial step maps each plain-text line from the Gherkin scenario to the corresponding Selenium automation method defined in the POM (search_page.py).

Step 5: Configuring the Environment and Executing Tests

I configured WebDriver lifecycle hooks in features/environment.py to automatically manage the setup and teardown of the browser session before and after each test scenario. Once configured, I executed the full BDD test suite by running the behave command in the terminal within the virtual environment.

(Insert screenshot of successful terminal execution showing 0 failed tests here)

```
iammin@iammindeMacBook-Air Week8 % python3 -m venv venv
iammin@iammindeMacBook-Air Week8 % source venv/bin/activate
(venv) iammin@iammindeMacBook-Air Week8 % pip install behave selenium pytest
```